

Báo cáo Reflection: Dự án AI Tour Guide

Môn học: Tư duy tính toán - CSC10014 - HK II năm 2026

Nhóm: GROUP02 - Lớp 24CTT6

Dự án: AI Tour Guide - web app hướng dẫn tham quan thông minh cho Đại Nội Huế

1. Cơ sở reflection và cách nhóm tự đánh giá

Theo nội dung bài học [LN11-Reflection.pdf](#), reflection không chỉ là nhìn lại xem sản phẩm đã chạy được hay chưa, mà là quá trình suy nghĩ lại toàn bộ cách nhóm giải quyết vấn đề, đánh giá điểm mạnh - điểm yếu, tìm nguyên nhân gốc của lỗi, rồi biến các bài học đó thành hành động cụ thể. Vì vậy, báo cáo này tập trung vào bốn câu hỏi chính:

1. Nhóm đã phân rã, thiết kế, hiện thực và debug bài toán như thế nào?
2. Giải pháp hiện tại có hiệu quả, đúng, sáng tạo và đủ đơn giản chưa?
3. Sai lầm nghiêm trọng nhất nằm ở kỹ thuật, quy trình hay cách ra quyết định?
4. Nếu làm lại, nhóm sẽ thay đổi chiến lược, abstraction và checklist làm việc ra sao?

Để tránh reflection hời hợt kiểu "sản phẩm chạy được là xong", nội dung dưới đây đối chiếu trực tiếp với commit history, cấu trúc code và luồng hoạt động thực tế của dự án.

2. Tổng quan sản phẩm đã xây dựng

AI Tour Guide là một ứng dụng web hỗ trợ khách tham quan tìm hiểu các di tích, hiện vật và lộ trình trong Đại Nội Huế. Người dùng có thể chọn điểm tham quan, hỏi bằng văn bản, gửi ảnh, ghi âm, xem bản đồ, nhận gợi ý điểm tiếp theo, đọc blog, đánh giá nội dung và thực hiện luồng mua vé mô phỏng qua VNPay.

Cấu trúc hiện tại gồm:

- **frontend/**: React 18 + Vite, giao diện chính, dashboard tham quan, chat, voice, bản đồ, blog, rating, payment, game và photo booth.
- **backend/**: FastAPI unified backend, gom các API vision, voice, chat, map, game, blog, payment, rating, feedback và health check.
- **migrations/**: Alembic migrations cho PostgreSQL, bao gồm bảng artifacts, locations, feedback, chat history, graph, blog, payment, ratings.
- **scripts/**: seed data, build graph, tạo báo cáo feedback và kiểm thử dữ liệu.
- **docs/**: tài liệu kiến trúc, demo script, troubleshooting và prompt cho photo booth.
- **backend/tests/** và **frontend/scripts/**: test API, unit test backend, test contract và các script kiểm thử frontend.

Luồng chính của hệ thống:

```
Người dùng mở web
-> chọn địa điểm hoặc vào dashboard
-> gửi text / ảnh / giọng nói
-> frontend gọi /api/v1/chat/unified hoặc các API chuyên biệt
-> backend validate ngôn ngữ, kích thước text, ảnh, audio, tọa độ, artifact_id
```

-> nếu có audio thì chạy STT, nếu có ảnh thì chạy Vision
-> tìm context trong PostgreSQL theo artifact_id, tên hiện vật, lịch sử hội thoại hoặc kết quả Vision
-> ưu tiên trả lời từ DB/cache/template khi đủ dữ liệu
-> chỉ gọi LLM khi cần diễn giải tự nhiên hoặc trả lời follow-up
-> frontend hiển thị câu trả lời, thông tin hiện vật, các bước xử lý, đọc bằng Web Speech API và cho phép feedback

3. Timeline phát triển

Dự án phát triển theo nhiều giai đoạn rõ ràng:

- **07/05/2026 - 11/05/2026:** Khởi tạo React project, camera scanning, image compression, API integration, backend ban đầu và màn hình chọn tính năng.
- **12/05/2026 - 19/05/2026:** Kết nối database, merge nhiều frontend/backend, thêm hướng dẫn chạy, thử cấu trúc mới, gộp hai hướng xử lý voice và image.
- **20/05/2026 - 27/05/2026:** Xây UI mới, sửa voice/autoplay, cập nhật RAG, giảm lỗi ảo giác khi voice rỗng và thêm phản hồi khi ảnh không rõ hoặc không tìm thấy.
- **01/06/2026 - 04/06/2026:** Phát triển map, thay map ảnh bằng icon 2D trên bản đồ thật, thêm roadmap, gợi ý điểm tiếp theo và game chơi cùng nhau.
- **10/06/2026 - 16/06/2026:** Làm giàu mô tả hiện vật, cải thiện pipeline LLM, thêm blog, tối ưu prompt AI, cải thiện TTS, liên kết map với chat, thêm draft cuối tour và gallery.
- **17/06/2026 - 20/06/2026:** Tối ưu tốc độ voice, thêm check-in, xử lý ảnh tốt hơn, tối ưu UI performance, sửa bug frontend và cải thiện mobile.
- **21/06/2026 - 22/06/2026:** Thêm payment, rating, food & drink, gallery, migrations mới và luồng VNPay popup.
- **25/06/2026 - 26/06/2026:** Bổ sung unit test backend, test plan và review trước demo.

Nhìn theo timeline này, nhóm đã đi từ prototype nhận diện ảnh sang một hệ thống nhiều module. Điểm tích cực là nhóm phản ứng nhanh với lỗi và liên tục mở rộng tính năng. Điểm yếu là nhiều commit có tính "fix bug", "merge", "improve UI" chung chung, cho thấy quá trình phát triển có lúc thiếu task boundary rõ và thiếu mô tả kỹ thuật đủ chi tiết.

4. Phân bổ thời gian và đánh giá quá trình

4.1. Phân rã bài toán và thiết kế

Ước tính nhóm dành khoảng **15% thời gian** cho phân rã và thiết kế. Bài toán ban đầu được chia thành các mảng tương đối đúng: frontend, backend API, nhận diện ảnh, voice/chat, dữ liệu hiện vật, bản đồ, game, blog và payment. Đây là cách decomposition hợp lý vì mỗi phần có đầu vào, đầu ra và trách nhiệm riêng.

Tuy nhiên, thiết kế ban đầu chưa đủ chặt ở hai điểm:

- Frontend chưa chọn sớm một abstraction routing chuẩn như React Router. File `App.jsx` hiện tự quản lý view bằng `useState`, `window.history`, query string và path parsing thủ công cho nhiều màn hình như dashboard, destination, blog, payment, game.
- State quan trọng của trải nghiệm tham quan và hội thoại chưa được quy hoạch đồng bộ ngay từ đầu. Backend đã có `ConversationMemory` lưu `chat_turns` trong PostgreSQL, nhưng frontend chưa có API khôi phục đầy đủ lịch sử chat khi reload.

- Ý tưởng về ứng dụng ban đầu còn sơ sài, chưa giải quyết được pain point của người dùng và nhận nhiều góp ý cải tiến của giảng viên để ứng dụng tốt hơn và có sự khác biệt với các nền tảng AI chatbot cơ bản khác.

4.2. Viết code

Ước tính nhóm dành khoảng **45% thời gian** cho coding. Giai đoạn này tạo ra nhiều module có giá trị:

- Unified backend FastAPI trong `backend/main.py`.
- Unified chat endpoint `/api/v1/chat/unified`.
- `UnifiedOrchestrator` điều phối STT, Vision, DB retrieval, memory, LLM và TTS.
- Vision service dùng Gemini, validate ảnh, optimize ảnh, retry lỗi tạm thời, fallback nhiều API key và rerank bằng tên, đặc trưng ảnh, GPS.
- LLM fallback provider thử nhiều backend theo thứ tự.
- Map service lập tour bằng greedy search theo khoảng cách và gọi OSRM để lấy đường đi.
- Frontend dashboard, chat, map, blog, game, rating, payment và Web Speech TTS chunking.

Điểm mạnh là nhóm không chỉ gọi AI trực tiếp, mà đã xây một pipeline có kiểm soát: validate đầu vào, dùng DB trước, cache câu trả lời, timeout provider và ghi nhận feedback. Điểm yếu là nhiều tính năng được thêm nhanh, làm frontend trở nên rộng và khó kiểm soát hơn.

4.3. Debug, tích hợp và sửa conflict

Ước tính nhóm dành khoảng **45% thời gian** cho debug và tích hợp. Tỷ lệ này cao hơn mong muốn, phản ánh đúng vấn đề trong quy trình:

- Nhiều nhánh và pull request được merge liên tục.
- Một số commit chỉ mô tả chung như "fix bug frontend", "Improve UI", "review before demo".
- Các tính năng chạm vào cùng luồng màn hình chính nên dễ phát sinh conflict ở `App.jsx`, CSS và service gọi API.
- AI provider có độ trễ, quota và lỗi mạng nên việc debug không chỉ là bug code mà còn là bug tích hợp hệ thống.
- Các chức năng làm xong không được test kĩ trước khi gộp vào code chính nên khi làm sang tính năng khác thì gặp vấn đề ở tính năng trước đó và phải tốn nhiều thời gian để debug.

Đây là phần nhóm lãng phí thời gian nhiều nhất. Nếu có task board, PR nhỏ hơn, code review nghiêm túc và checklist tích hợp, thời gian debug có thể giảm đáng kể.

5. Đánh giá chiến lược và abstraction

5.1. Chiến lược đúng: DB-first, cache-first, LLM-last

Quyết định tốt nhất của nhóm là không để LLM trở thành nguồn sự thật duy nhất. Trong `UnifiedOrchestrator`, hệ thống tìm hiện vật theo `artifact_id`, lịch sử hội thoại, tên hiện vật, Vision result hoặc query text. Nếu có dữ liệu rõ, backend có thể trả lời bằng cache, template hoặc DB direct trước khi gọi LLM.

Lợi ích:

- Giảm hallucination vì câu trả lời bám vào dữ liệu PostgreSQL.

- Giảm chi phí và độ trễ do tránh gọi LLM cho câu hỏi lặp lại hoặc câu hỏi đơn giản.
- Dễ debug hơn vì response có `answer_source` và `processing_steps`.

Đây là ví dụ tốt của abstraction trong tư duy tính toán: tách "tìm dữ liệu đáng tin cậy" khỏi "diễn giải ngôn ngữ tự nhiên".

5.2. Chiến lược đúng: hợp nhất API đa phương thức

Việc gom text, image và voice vào `/api/v1/chat/unified` giúp frontend đơn giản hơn so với việc tự ghép nhiều endpoint riêng. Backend có thể quyết định khi nào cần STT, khi nào cần Vision, khi nào chỉ dùng text. Đây là decomposition tốt vì frontend gửi intent, còn backend điều phối pipeline xử lý.

Tuy nhiên, unified endpoint cũng làm orchestrator phình to. File `unified_orchestrator.py` đang gánh nhiều trách nhiệm: nhận diện ngữ cảnh, gọi provider, phân loại query, cache, prompt, memory, fallback và format response. Nếu tiếp tục mở rộng, nên tách thành các service nhỏ hơn như `InputResolver`, `ArtifactContextResolver`, `AnswerStrategy`, `ProviderFallbackPolicy`.

5.3. Chiến lược chưa tốt: routing frontend tự viết

Một sai lầm thiết kế rõ ràng là routing thủ công trong `App.jsx`. Hiện code phải tự xử lý:

- `window.history.pushState` và `replaceState`.
- `popstate`.
- query `view`, `destination`, `frame`, `code`.
- path `/blog`, `/blog/new`, `/blog/:slug`.
- lazy preload cho từng view.

Cách này vẫn chạy, nhưng khi thêm blog, payment result, destination detail, phone preview và game join, logic điều hướng trở nên khó kiểm thử và khó mở rộng. Nếu dùng React Router từ đầu, nhóm có thể có route tree rõ ràng hơn, loader/action tốt hơn và deep link ổn định hơn.

5.4. Chiến lược còn thiếu: quản lý state dài hạn

Backend đã lưu lịch sử hội thoại trong PostgreSQL qua `ConversationMemory`, nhưng frontend chưa tận dụng triệt để để khôi phục phiên sau reload. Một số trạng thái trải nghiệm vẫn phụ thuộc vào local state, localStorage hoặc sessionStorage. Điều này tạo ra khoảng cách giữa "dữ liệu backend còn" và "UI người dùng thấy lại được".

Vấn đề này không làm sập demo ngay, nhưng ảnh hưởng lớn đến trải nghiệm thật: khách tham quan có thể reload trang, đổi thiết bị hoặc mất kết nối tạm thời. Với một tour guide thực tế, trạng thái chuyển tham quan nên được lưu theo `session_id`/user/trip thay vì chỉ nằm trong trình duyệt.

6. Phân tích hiệu suất và trade-off

6.1. Điểm đã làm tốt

Backend có nhiều cơ chế kiểm soát hiệu suất:

- STT timeout 20 giây, Vision timeout theo cấu hình, LLM timeout 30 giây và follow-up timeout 25 giây.
- LLM fallback provider giới hạn mỗi provider 10 giây.
- Frontend abort request unified chat sau 35 giây.

- Vision optimize ảnh trước khi gọi API.
- Cache câu trả lời và cache intro để tránh gọi LLM lặp lại.
- Web Speech API ở frontend chia văn bản thành chunk khoảng 220 ký tự để pause/resume ổn định hơn.
- Map planning dùng greedy search đơn giản, đủ nhanh cho dataset hiện tại.

Những quyết định này phù hợp với yêu cầu demo và dataset vừa phải. Nhóm ưu tiên độ phản hồi và khả năng chạy được trên máy local hơn là tối ưu thuật toán phức tạp.

6.2. Trade-off còn tồn tại

Các trade-off chính:

- **Độ chính xác AI vs độ trễ:** Vision, STT và LLM phụ thuộc provider bên ngoài như Gemini/Groq, nên độ trễ và quota không hoàn toàn kiểm soát được.
- **Greedy route vs tối ưu toàn cục:** Tour planning chọn điểm gần nhất kế tiếp. Cách này nhanh và dễ giải thích, nhưng không đảm bảo lịch trình tối ưu như TSP/constraint optimization.
- **Web Speech API vs backend TTS:** Dùng Web Speech API giảm tải backend và dễ chạy demo, nhưng giọng đọc phụ thuộc trình duyệt/thiết bị.
- **In-memory game room vs persistent room:** Game room chạy nhanh và đơn giản, nhưng nếu backend restart thì trạng thái phòng có thể mất.
- **Manual routing vs tốc độ phát triển:** Tự routing giúp triển khai nhanh lúc đầu, nhưng chi phí bảo trì tăng mạnh khi số màn hình tăng.

Reflection quan trọng ở đây là không có trade-off nào "sai tuyệt đối"; vấn đề là nhóm chưa ghi lại các trade-off này thành decision log, nên về sau khó biết cái gì là lựa chọn có chủ ý và cái gì là nợ kỹ thuật.

7. Phân tích lỗi và nguyên nhân gốc

7.1. Lỗi kỹ thuật nghiêm trọng nhất

Lỗi kỹ thuật nghiêm trọng nhất là **thiếu cơ chế khôi phục trạng thái trải nghiệm người dùng một cách nhất quán**.

Triệu chứng:

- Người dùng reload trang có thể mất trạng thái UI hoặc luồng hội thoại đang thấy.
- Backend vẫn có thể lưu `chat_turns`, nhưng frontend chưa có luồng fetch lịch sử và dựng lại UI đầy đủ.
- Routing thủ công khiến một số màn hình phụ phụ thuộc vào query/path state để lệch với React state.

Nguyên nhân gốc:

- Nhóm tập trung làm tính năng nhìn thấy được trước: chat, voice, image, map, UI, blog, payment.
- Chưa thiết kế sớm mô hình "tour session" làm trung tâm.
- Chưa có checklist yêu cầu mỗi feature phải trả lời: dữ liệu nào cần persist, reload có khôi phục được không, deep link có mở đúng view không.

7.2. Lỗi quy trình nghiêm trọng nhất

Lỗi quy trình nghiêm trọng nhất là **Bắt tay vào code khi thiết kế còn sơ sài, chưa rõ ràng và phân chia task, review chưa đủ chặt**.

Biểu hiện:

- Ứng dụng bị thay đổi thiết kế và sửa nhiều lần. Các tính năng được thêm vào mà chưa được lên kế hoạch rõ ràng.
- Nhiều commit sửa bug tổng quát, thiếu mô tả root cause.
- Nhiều thành viên cùng chạm vào UI chính và API service.
- Thời gian debug/conflict chiếm tỷ lệ lớn.
- Một số quyết định quan trọng như routing, persistence, fallback provider không được ghi thành checklist hoặc ADR ngắn.

Nguyên nhân gốc:

- Thiếu kinh nghiệm về thiết kế và tổ chức dự án.
- Bị thu hút bởi code mà không tập vào thiết kế và tổ chức hệ thống ban đầu.
- Giao tiếp kỹ thuật chủ yếu theo tin nhắn ngắn, dễ trôi ngử cảnh.
- Chưa có quy ước PR nhỏ, reviewer rõ, tiêu chí merge rõ.
- Chưa tách ownership theo module đủ mạnh.

8. Những kỹ năng tư duy tính toán đã áp dụng

8.1. Decomposition

Nhóm chia bài toán thành nhiều module:

- Input: text, audio, image.
- Processing: STT, Vision, retrieval, LLM, TTS.
- Data: artifacts, locations, graph, ratings, feedback, blog, payment.
- UI: home, dashboard, map, chat, destination detail, blog, game, checkout.
- Reliability: validation, rate limit, timeout, error handler, request id, tests.

Decomposition này giúp các thành viên có thể phát triển song song. Tuy nhiên, một số boundary chưa sạch, đặc biệt là frontend routing và orchestrator backend.

8.2. Pattern recognition

Nhóm nhận ra nhiều pattern lặp:

- Câu hỏi lặp về hiện vật nên cache.
- Câu hỏi đơn giản nên dùng template/DB direct.
- Ảnh mờ hoặc không liên quan phải trả lời rõ thay vì sinh câu chào sai.
- Audio rỗng cần phát hiện sớm để tránh hallucination.
- Provider AI có thể timeout/quota, cần fallback.

Các commit ngày 27/05/2026 về voice rỗng và ảnh không rõ là ví dụ tốt của debugging reflection: nhóm phát hiện pattern lỗi rồi thêm nhánh xử lý cụ thể.

8.3. Abstraction

Các abstraction tốt:

- **BaseLLM**, **BaseSTT**, **BaseTTS** cho provider AI.

- **FallbackLLMProvider** để thay đổi provider mà không sửa toàn bộ orchestrator.
- **ConversationMemory** để tách lưu lịch sử khỏi logic chat.
- Repository cho artifact lookup.
- API service frontend gom các request backend vào một nơi.

Các abstraction cần cải thiện:

- Routing frontend nên chuyển sang route abstraction chuẩn.
- Orchestrator nên tách nhỏ chiến lược chọn context và chiến lược chọn cách trả lời.
- Game room state nên tách khỏi RAM nếu muốn dùng thực tế.

8.4. Algorithmic thinking

Một số thuật toán/heuristic đã dùng:

- Haversine distance để tính khoảng cách giữa tọa độ.
- Greedy search cho tour planning.
- Scoring gợi ý điểm tiếp theo dựa trên khoảng cách và quan hệ trong knowledge graph.
- Vision reranking dựa trên confidence, alias match, visual feature match và GPS.
- Text normalization để xử lý tiếng Việt không dấu/có dấu.
- Chunking câu cho Web Speech API.

Các thuật toán này không quá phức tạp, nhưng phù hợp với dataset và yêu cầu demo. Bài học là thuật toán đơn giản, giải thích được và có fallback thường hiệu quả hơn trong sản phẩm demo so với thuật toán tối ưu nhưng khó tích hợp.

9. Đánh giá kết quả so với mục tiêu ban đầu

9.1. Mục tiêu ban đầu

Nhóm đặt mục tiêu xây dựng một hướng dẫn viên ảo mượt mà, hỗ trợ khách tham quan bằng nhiều phương thức nhập, có dữ liệu lịch sử đáng tin cậy, có bản đồ/lộ trình và trải nghiệm phù hợp thiết bị di động.

9.2. Kết quả đạt được

Các mục tiêu đã đạt khá tốt:

- Có frontend React/Vite và backend FastAPI thống nhất.
- Có chat đa phương thức text, ảnh, voice.
- Có RAG DB-first với PostgreSQL.
- Có nhận diện ảnh bằng Gemini Vision và xử lý ảnh không hợp lệ/không rõ.
- Có STT bằng Groq và LLM fallback.
- Có Web Speech TTS ở frontend.
- Có map, route, tour planning, next suggestion.
- Có blog, gallery, food & drink, rating, feedback, payment mock/VNPay flow.
- Có migrations và test backend/frontend ở nhiều mức.

9.3. Điểm chưa đạt

Các điểm chưa đạt hoặc chưa đủ chắc:

- Trạng thái tour/chat chưa khôi phục tốt sau reload hoặc đổi thiết bị.
- Frontend routing chưa đủ chuẩn cho ứng dụng nhiều màn hình.
- Game room state chưa persistent.
- Vision/STT vẫn phụ thuộc provider ngoài và quota.
- Chưa có automated end-to-end test cho toàn bộ luồng demo từ frontend đến backend.
- Chưa có decision log để ghi lại lý do chọn kiến trúc.

10. Nếu làm lại dự án, nhóm sẽ thay đổi gì?

10.1. Thay đổi quy trình

Quan trọng nhất, nhóm sẽ tập trung thiết kế, phân tích rõ hệ thống, các chức năng cần thực hiện, các chức năng gì giải quyết các pain point gì của người dùng. Phân rã các vấn đề thật tốt để phân chia nhiệm vụ tốt hơn. Áp dụng các quy trình làm việc nhóm thay vì chỉ làm theo cảm tính. Nhóm sẽ áp dụng các quy tắc:

- Mỗi task có owner, phạm vi file và tiêu chí hoàn thành rõ.
- Mỗi PR không nên trộn UI, backend, migration và refactor nếu không bắt buộc.
- Commit message phải nêu hành động và lý do, ví dụ `fix(chat): restore active artifact after reload`.
- Trước khi merge phải có reviewer chạy checklist: build, test, reload flow, mobile view, API error state.
- Mỗi tuần ghi learning log: lỗi lớn nhất, nguyên nhân gốc, cách sửa, bài học tái sử dụng.

10.2. Thay đổi chiến lược

Nếu làm lại, nhóm sẽ thiết kế quanh ba khái niệm trung tâm ngay từ đầu:

1. **TourSession**: lưu trạng thái chuyến tham quan, điểm đang chọn, điểm đã đi, ngôn ngữ, session_id.
2. **ConversationThread**: lưu và khôi phục hội thoại theo session.
3. **RouteState**: lưu trạng thái điều hướng bằng React Router thay vì tự quản lý bằng query thủ công.

Cách này giúp frontend reload vẫn dựng lại đúng trạng thái từ backend.

10.3. Thay đổi implementation

Các thay đổi cụ thể:

- Dùng React Router cho route `/, /destinations/:id, /tour, /blog, /blog/:slug, /game/join/:code, /payment/result`.
- Thêm API `GET /api/v1/chat/history?session_id=...` để frontend khôi phục `chat_turns`.
- Lưu trip/passport/journal vào PostgreSQL theo session thay vì phụ thuộc local browser state.
- Lưu game room vào Redis hoặc PostgreSQL nếu cần multiplayer ổn định.
- Tách **UnifiedOrchestrator** thành các lớp nhỏ hơn để dễ test.
- Viết ADR ngắn cho các quyết định lớn: DB-first RAG, Web Speech TTS, provider fallback, routing, persistence.

11. Action items cụ thể sau reflection

Thay vì viết chung chung "lần sau cẩn thận hơn", nhóm rút ra các checklist cụ thể:

1. **Checklist persistence:** Mỗi state quan trọng phải trả lời được: lưu ở đâu, reload có khôi phục được không, đổi thiết bị có dùng lại được không?
2. **Checklist AI provider:** Mỗi provider call phải có timeout, lỗi người dùng dễ hiểu, logging và fallback nếu khả thi.
3. **Checklist routing:** Mỗi màn hình mới phải có URL rõ, back/forward hoạt động và deep link mở lại được.
4. **Checklist input validation:** Text, ảnh, audio, tọa độ, session_id và artifact_id phải validate trước khi gọi AI.
5. **Checklist test:** Với mỗi feature chính cần ít nhất một test unit/service và một test API hoặc contract.
6. **Checklist reflection hằng tuần:** Ghi lại lỗi nghiêm trọng nhất, pattern/abstraction bị bỏ lỡ, giải pháp đơn giản hơn nếu có và kỹ năng cần luyện tiếp.

12. Kỹ năng cần cải thiện sau đề án này

Vì môn học tập trung vào tư duy giải quyết vấn đề, phân rã bài toán, làm việc nhóm và chủ yếu về tư duy thay vì code nên các kỹ năng liên quan đến nội dung môn học và đề án cần cải thiện là:

- Cách thiết kế, xây dựng 1 dự án hoàn chỉnh.
- Cách nhìn nhận vấn đề, tư duy tập trung vào người dùng thay vì chỉ nhồi nhét kĩ thuật.
- Cách phân chia công việc, giao nhiệm vụ hay quản lí dự án và cách sử dụng git.
- Cách giao tiếp và các phương pháp làm việc nhóm
- Cách sử dụng AI và ra lệnh cho AI.

13. Kết luận

Dự án AI Tour Guide đạt được nhiều tính năng hơn một prototype cơ bản: có AI đa phương thức, RAG theo dữ liệu hiện vật, fallback provider, bản đồ, game, blog, rating, payment và test. Điểm mạnh nhất của nhóm là khả năng tích hợp nhanh nhiều thành phần và cải thiện liên tục sau khi gặp lỗi.

Tuy nhiên, reflection cho thấy vấn đề lớn nhất không nằm ở một bug đơn lẻ, mà nằm ở cách thiết kế state và quy trình phát triển. Nhóm đã giải quyết tốt nhiều bài toán cục bộ, nhưng chưa đủ sớm xây nền tảng routing, persistence và decision log cho một hệ thống mở rộng. Bài học quan trọng nhất là: trong một dự án có AI, frontend, backend, database và nhiều thành viên, tư duy tính toán không chỉ là chia nhỏ bài toán để code nhanh, mà còn là chọn abstraction đúng, ghi lại quyết định, kiểm tra trade-off và biến lỗi thành checklist hành động cụ thể.